Arhitektura računara i operativni sistemi

Laboratorijske vežbe 6

1. Sistemski poziv fork()

Sistemski pozivi omogućavaju procesima da zatraže usluge od kernela operativnog sistema. Sistemski poziv **`fork()`** se koristi za kreiranje kopije trenutnog procesa. Naime, pozivom funkcije *fork* kreira se kopija procesa unutar kog je *fork* pozvan - kopira se brojač naredbi trenutnog procesa, njegovi memorijski segmenti - *stack*, *heap*, *text*, *data* (podsetite se adresnog prostora procesa sa petog termina vežbi). Proces unutar kog je pozvan *fork* naziva se i ***roditelj-proces*** (engl. *parent process*), dok se novokreirani proces naziva ***dete-proces*** (engl. *child process*).

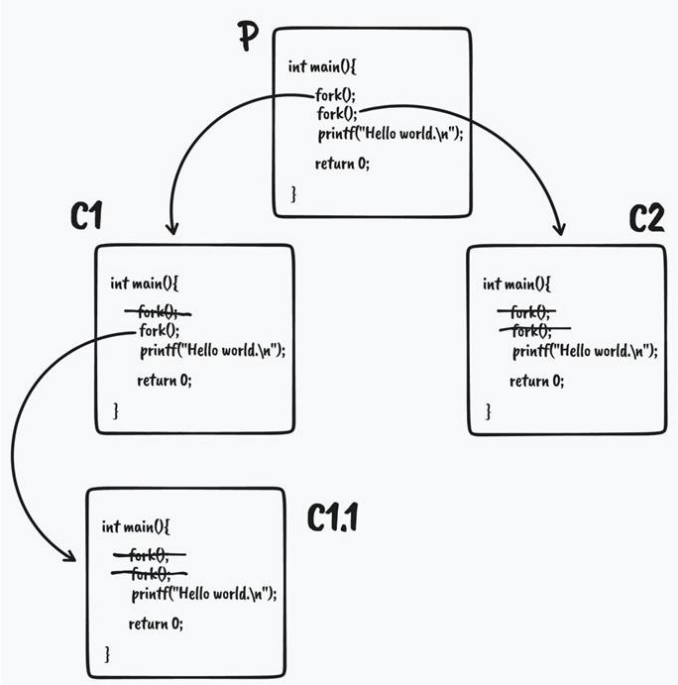
Dete-proces nastavlja da se izvršava **od linije u kojoj je funkcija `fork()` pozvana**. Posmatrajmo sledeći program:

```
#include <stdio.h>
#include <unistd.h>

int main(){

    fork();
    fork();
    printf("Hello world.\n");

return 0;

}
```

Slika 1 - fork()

Funkcija **`fork()`** je pozvana dvaput. Ako ovaj program pokrenemo, *Hello world* će se ispisati četiri puta. Kako se ovo izvršava, prikazano je na slici 2 (*Slika 2*). Dakle, u procesu **P**, prvim pozivom funkcije **`fork()`**, kreira se novi proces (**C1**), koji nastavlja sa izvršavanjem od sledeće linije koda. Proces **S1** takođe poziva funkciju **`fork()`**, koja kreira novi proces - **C1.1**.

Vratimo se ponovo na proces **P**. Drugi poziv **`fork()`** je kreirao proces **C2** u kom se na kraju izvršava samo linija u kojoj se poziva funkcija **`printf`**.



Slika 2 - Izvršavanje programa sa slike 1

Posmatrajte program na slici 3 (*Slika 3*). Funkcija **`fork()`** vraća vrednost tipa **`pid_t`**. Ukoliko se radi o roditeljskom procesu, vrednost koju vraća ova funkcija biće jednaka identifikatoru (*PID*) procesa-deteta, koji je upravo kreiran. Ova vrednost će biti **veća od 0**. Ukoliko je reč o detetu-procesu, vrednost koju vraća funkcija **`fork()`** **biće 0**, a ukoliko je došlo do greške, funkcija će vratiti -1. Dete-proces obično dobija prvi slobodan PID, pa će to na mnogim sistemima biti PID koji je za jedan veći od identifikatora roditelja. Prilikom jednog pokretanja programa sa slike 1, roditelj proces imao je PID 122825, proces C1 122826, C1.1 122827, a C2 122828. Identifikatore procesa možemo saznati i pomoću funkcija **`getpid()`**, za identifikator trenutnog procesa, i **`getppid()`**, za identifikator roditeljskog procesa, kao što je prikazano na slici 3. U okviru deteta-procesa poziva se funkcija **`exit`**, koja predstavlja **sistemski poziv koji prekida trenutni proces**. Ovoj funkciji prosleđujemo neki izlazni kod - u ovom slučaju - 123. Roditelj-proces poziva blokirajuću funkciju **`wait`**, koja predstavlja **sistemski poziv koji čeka završetak deteta-procesa**. Ukoliko roditelj-proces ima više dece, funkcija **`wait`** vraća PID deteta-procesa koje se prvo izvršilo. Funkciji **`wait`** prosleđujemo adresu promenljive u koju će se upisati izlazni status deteta-procesa.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

int main(){

    pid_t pid;
    int exitStatus;

    switch(pid = fork()){
        case -1:
        printf("Fork error!\n");

        case 0:

        printf("Child: This is child process with PID %d\n", getpid());

        printf("Child: My parent's PID is %d\n", getppid());
        printf("Child: Bye!\n");
        sleep(10);
        exit(123);

        default:
        printf("Parent: This is parent process with PID %d\n", getpid());
        printf("Parent: My child's PID is %d\n", pid);
        printf("Parent: I'm waiting for my child...\n");
        wait(&exitStatus);
        printf("Parent: My child exited with exit status %d\n", WEXITSTATUS(exitStatus));
        printf("Parent: Bye!\n");
    }

    return 0;
}
```

Slika 3 – Prikaz PID (fork.c fajl)

Pošto će se u varijabli **`exitStatus`** sačuvati i dodatne informacije (da li je proces uspešno završen, kojim signalom je prekinut itd.), da bismo iz ove promenljive pročitali samo izlazni kod deteta-procesa (onaj koji je prosleđen funkciji **`exit`**), koristi se makro **`WEXITSTATUS`**.

```
[student@fedora fork_exec]$ ./fork
Parent: This is parent process with PID 2093
Parent: My child's PID is 2094
Parent: I'm waiting for my child...
Child: This is child process with PID 2094
Child: My parent's PID is 2093
Child: Bye!
Parent: My child exited with exit status 123
Parent: Bye!
```

Slika 4 - Izvršavanje programa sa slike 3

Kompajlirajte i pokrenite program na slici 3, a uporedo pokrenite i naredbu **`ps aux`**. Primetićete da će se pokretanjem ovog programa zaista kreirati dva procesa:

```
student     2140  0.0  0.0   2608   944 pts/1    S+   19:36   0:00 ./fork
student     2141  0.0  0.0   2608    88 pts/1    S+   19:36   0:00 ./fork
```

Slika 5 - ps aux

2. Sistemski poziv execve()

Sistemski poziv **`execve()`** započinje novi proces. Nakon izvršanja ovog sistemskog poziva, proces iz kojeg se poziva sistemski poziv se prekida, a njegov PID zauzima novi proces sa svojim *stack*-om, *heap*-om, *data* i *text* sekcijom. Poziv **`execve()`** se najčešće koristi u kombinaciji sa sistemskim pozivom **`fork()`**, kako bi se dete-proces zamenio nekim drugim procesom.

Potpis funkcije **`execve()`** dat je u nastavku:

```
int execve(const char *pathname, char *const argv[], char *const envp[]);
```

Argument **pathname** sadrži putanju do programa čiji proces treba da bude započet. Prosleđena putanja može biti apsolutna ili relativna u odnosu na radni direktorijum. Argument **argv** predstavlja argumente komandne linije koji će biti prosleđenim novom procesu. Poslednjni argument **envp** predstavlja promenljive okruženja (engl. e*nvironment variables*). Ukoliko se uspešno izvrši, funkcija ne vraća nikakvu vrednost, a ukoliko je došlo do greške, funkcija vraća vrednost -1.

Postoji čitava familija funkcija koje počinju sa ***exec*** i koje su implementirane uz pomoć sistemskog poziva **`execve()`**. Potpisi nekih od funkcija dati su u nastavku:

```
int execl(const char *path, const char *arg, ...);
```

```
int execlp(const char *file, const char *arg, ...);
```

```
int execv(const char *path, char *const argv[]);
```

```
int execvp(const char *file, char *const argv[]);
```

Većina exec funkcija kao prvi argument prima putanju do programa. Međutim, funkcije **`execlp()`** i **`execvp()`** kao prvi argument prihvataju naziv fajla. Program sa prosleđenim nazivom se potom pretražuje po direktorijumima definisanim u promenljivoj okruženja PATH. Umesto da koriste niz za prosleđivanje argumenata komandne linije, funkcije **`execl`** i **`execlp`** dozvoljavaju da se argumenti komandne linije prosleđuju kao zasebne niske, što znači da ćemo u ovom delu proslediti onoliko argumenata koliko imamo argumenata komandne linije.

Kompajlirajte i pokrenite program **forkAndExec.c** sa različitim potpisima funkcija ovog sistemskog poziva i pokušajte na taj način da pokrenete procese različitih programa.

3. Makefile

Makefile su fajlovi za organizaciju kompilacije koda. Koriste se za određivanje onih delova programa koje je potrebno rekompajlirati. Najčešće se koriste za kompajliranje C i C++ koda. **Make** je alat iz GNU porodice alata, čiji su tvorci Ričard Stolman i Roland Makgrat.

Opšta sintaksa jednog pravila u okviru *makefile*-a data je u nastavku:

```
targets: prerequisites
    command
    command
    command
```

Slika 6 - Struktura Makefile-a

- *targets* - predstavlja nazive fajlova
- *command* - niz koraka za dobijanje *target*-a i oni moraju da započinju tabulatorom
- *prerequisites* (zavisnosti) - predstavlja nazive fajlova koji moraju da postoje kako bi se komande za *target* pokrenule.

Napisaćemo Makefile za *reader-writer* problem obrađen u terminu 6.

```
read: read.c
    gcc -o read read.c
write: write.c
    gcc -o write write.c
```

Slika 7 - Makefile za reader-writer

Izvršenje komandi određenog *target*-a pokrećemo naredbom:

`make naziv_target-a`

Za izvršenje komandi oba *target*-a koristićemo naredbu:

```
make read write
```

Nakon izvršenja ove naredbe, dobijamo dva fajla: *read* i *write*. Ovi fajlovi su rezultat kompajliranja uz pomoć *gcc* kompajlera.

Ukoliko opet izvršimo **make** naredbu bez da smo izmenili fajlove *read.c* i *write.c*, dobićemo poruku:

```
└─$ make read write
make: 'read' is up to date.
make: 'write' is up to date.
```

Slika 8 - Ponovno izvršenje make bez izmene .c fajlova

Razlog je što smo kao zavisnosti naveli fajlove *read.c* i *write.c* i make neće izvršiti komande ukoliko je datum poslednje izmene zavisnosti stariji od datuma kreiranja *target*-a. Ukoliko bismo izmenili neki od fajlova *read.c* ili *write.c* i onda izvršili naredbu **make**, došlo bi do ponovnog kompajliranja onog *target*-a čiji je fajl izmenjen.

4. Git

Git je sistem kontrole verzija koji je razvio Linus Torvalds 2005. godine. Primarno se koristi za verzionisanje koda, odnosno, praćenje promena nastalih u kodu i koordinisan rad više programera na istom projektu.

Nakon instalacije git-a, prvo je potrebno konfigurisati naše podatke koristeći sledeće dve naredbe:

```
git config --global user.name "username"
git config --global user.email "mail@mail.com"
```

Slika 9 – Konfiguracija git podataka o korisniku

Podaci koji se ovde unesu će se kasnije koristiti prilikom registracije na neku od platformi koje nam nude rad sa udaljenim repozitorijumima (npr. *Github, Gitlab, Bitbucket*, …).

Git projekte organizuje u, takozvane, repozitorijume. Za inicijalizaciju git repozitorijuma koristi se naredba

```
git init
```

Nakon uspešnog izvršenja ove naredbe, u direktorijumu se pojavljuje skriveni direktorijum **.git** u kojem se nalaze svi neophodni podaci za verzionisanje naših fajlova.

Kada izvršimo naredbu

```
git status
```

možemo videti trenutni status fajlova u repozitorijumu.

```
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        program.c

nothing added to commit but untracked files present (use "git add" to track)
```

Slika 10 - git status

Fajlovi mogu da se nalaze u dva stanja:

- *Tracked* – označava da je fajl dodat u repozitorijum
- *Untracked* – fajl se nalazi u direktorijumu, ali nije dodat u repozitorijum

Prilikom kreiranja novog repozitorijuma, svi fajlovi će biti u stanju *untracked*. Da bismo dodali pojedinačni fajl u repozitorijum, koristićemo sledeću naredbu:

```
git add naziv_fajla
```

ili ukoliko želimo da dodamo sve fajlove u repozitorijum koristimo naredbu:

```
git add .
```

Ukoliko sada izvršimo naredbu

```
git status
```

videćemo da su fajlovi prešli u stanje *tracked*. Ovim smo premestili fajlove u *Staging* okruženje.

U sledećem koraku potrebno preći iz *stage* faze u *commit* fazu. Dodavanjem *commit*-ova zapravo beležimo sve novonastale promene. Ovo je tačka čuvanja trenutne verzije projekta. Git nam omogućava da u bilo kom trenutku možemo da se vratimo ili izmenimo neki prethodni *commit*. Za dodavanje *commit*-a koristi se sledeća naredba:

```
git commit -m "Poruka commit-a"
```

4.1. Github

Github je veb platforma za pravljenje udaljenih git repozitorijuma. Kada napravimo git repozitorijum on se nalazi samo lokalno, na našem računaru. Github nam pruža mogućnost da naš repozitorijum postavimo na udaljeno skladište i tako učinimo naš repozitorijum vidljiv ostalim korisnicima Github-a. Moguće je pravljenje i privatnog repozitorijuma, koji bi onda bio vidljiv samo korisnicima kojima damo dozvolu.

Za pravljenje udaljenog repozitorijuma, prvo je potrebno napraviti nalog na Github-u. Nakon ovoga se klikom na dugme **New** otvara nova stranica za kreiranje repozitorijuma:

U polje **Repository name** stavljamo naziv našeg repozitorijuma. Nakon unosa imena možemo kreirati repozitorijum klikom na dugme **Create repository**.

Sada je potrebno dodati udaljeni repozitorijum u naš projekat. To radimo naredbom:

```
git remote add origin url_do_repozitorijuma.git
```

Nakon toga je potrebno da postavimo granu u kojoj ćemo raditi naredbom (ovde je korišćena *main* grana):

```
git branch -M main
```

Za otpremanje *commit*-ova iz lokalnog, na *main* granu udaljenog repozitorijuma koristimo naredbu:

```
git push -u origin main
```